Resumo – Dicionários

1. A ideia central

Imagine um cadastro de aluno com nome, nota e frequência. Com uma lista, ficaria assim:

```
aluno = ["Ana", 85, 92]
```

Funciona, mas o que significa `aluno[1]`? É a nota ou a frequência? E se precisarmos acrescentar email, matrícula, turma? As posições deixam de ser intuitivas.

Quando cada informação tem um **nome**, é mais natural acessar pelo nome. Esse nome é a **chave**, e a informação é o **valor**. Um **dicionário** é uma coleção de associações **chave** → **valor**.

```
aluno = {
    "nome": "Ana",
    "nota": 85,
    "frequencia": 92
}

print(aluno["nome"])
print(aluno["nota"])
```

```
Ana
85
```

	Lista	Dicionário
delimitador	colchetes `[ ]`	chaves `{ }`
acesso	por **posição**: `aluno[0]`	por **chave**: `aluno["nome"]`
cada item	um valor	um par `"chave": valor`
quando usar	vários valores do mesmo tipo	dados com significado próprio

Macete

Para criar: abre com `{`, cada item é `"chave": valor`, itens separados por **vírgula**, fecha com `}`. Para **acessar**, porém, usamos **colchetes**, igual à lista: `aluno["nome"]`. Chaves para criar, colchetes para acessar.

2. As operações fundamentais

Acessar

```
produto = {"nome": "Mouse", "preco": 120, "estoque": 5}
```

```python
print(produto["preco"])      # 120
print(produto["tamanho"])    # KeyError: 'tamanho'
```

Atenção

Acessar uma chave que não existe causa `KeyError` e interrompe o programa. É o equivalente, nos dicionários, ao `IndexError` das listas.

Alterar e adicionar: a mesma sintaxe

```python
produto = {"nome": "Mouse", "preco": 120, "estoque": 5}

produto["estoque"] = 10
produto["categoria"] = "Periferico"

print(produto)
```

```
{'nome': 'Mouse', 'preco': 120, 'estoque': 10, 'categoria': 'Periferico'}
```

Macete

Alterar ≠ adicionar, mas a linha de código é idêntica. O Python decide sozinho: se a chave **já existe**, o valor é atualizado (mesma chave, novo valor); se **não existe**, uma nova associação é criada e o dicionário cresce. Em questões de leitura de código, pergunte sempre: "essa chave já estava lá?"

Verificar se uma chave existe

```python
aluno = {"nome": "Ana", "nota": 85}

if "nome" in aluno:
    print(aluno["nome"])

print("email" in aluno)
print("Ana" in aluno)
```

```
Ana
False
False
```

Atenção

O operador `in` procura nas **chaves**, não nos valores. Por isso `"Ana" in aluno` é `False`: `"Ana"` é um valor, e não uma chave.

Remover

```python
aluno = {"nome": "Ana", "nota": 85, "frequencia": 92}

del aluno["frequencia"]

print(aluno)
```

```
{'nome': 'Ana', 'nota': 85}
```

Depois do `del`, a chave deixa de existir, o valor associado desaparece junto e acessá-la novamente gera erro.

Explorar: keys, values e items

```
aluno = {"nome": "Ana", "nota": 85}

print(aluno.keys())
print(aluno.values())
print(aluno.items())
```

```
dict_keys(['nome', 'nota'])
dict_values(['Ana', 85])
dict_items([('nome', 'Ana'), ('nota', 85)])
```

`keys()` mostra quais informações existem; `values()` mostra os dados armazenados; `items()` mostra as associações completas, cada uma como um par `(chave, valor)`.

3. Percorrendo um dicionário

```
estoque = {"mouse": 10, "teclado": 5, "monitor": 2}

for chave in estoque:
    print(chave, estoque[chave])

for valor in estoque.values():
    print(valor)

for produto, quantidade in estoque.items():
    print(produto, "->", quantidade, "unidades")
```

```
mouse 10
teclado 5
monitor 2
10
5
2
mouse -> 10 unidades
teclado -> 5 unidades
monitor -> 2 unidades
```

Macete

As três formas, em uma frase cada: `for chave in d` percorre as **chaves** (e `d[chave]` dá o valor); `for valor in d.values()` percorre só os **valores**; `for chave, valor in d.items()` percorre os **dois juntos**. Na dúvida, use `items()`: ele sempre serve. Repare que `items()` exige **duas** variáveis no `for`.

Macete

O dicionário é percorrido na **ordem em que as chaves foram inseridas**. Alterar o valor de uma chave existente **não** muda a posição dela. Mas remover uma chave com `del` e inseri-la de novo a manda para o **final**. Isso decide muitas questões de leitura de código.

4. Os padrões que resolvem quase tudo

Padrão 1: contagem (o mais cobrado)

Contar votos, palavras, letras: a chave é "o que estou contando" e o valor é "quantas vezes apareceu".

```
votos = ["Ana", "Carlos", "Ana", "Julia", "Carlos", "Ana"]
contagem = {}

for nome in votos:
    if nome in contagem:
        contagem[nome] = contagem[nome] + 1
    else:
        contagem[nome] = 1

print(contagem)
```

```
{'Ana': 3, 'Carlos': 2, 'Julia': 1}
```

Macete

A receita da contagem tem sempre a mesma pergunta: **"já vi essa chave?"** Se sim, soma 1. Se não, cria com 1. Nunca faça `contagem[nome] + 1` sem conferir antes: na primeira aparição a chave não existe e o programa quebra com `KeyError`. Para **somar valores** por chave (total vendido por produto, por exemplo), é a mesma receita, trocando o 1 pelo valor.

Padrão 2: a chave do maior valor

```
contagem = {"Ana": 3, "Carlos": 2, "Julia": 1}

vencedor = ""
maior = 0

for nome, total in contagem.items():
    if total > maior:
        maior = total
        vencedor = nome

print(vencedor, maior)
```

```
Ana 3
```

É o padrão "maior valor" das listas, guardando também a chave. Com `>`, em caso de empate vence quem foi inserido **primeiro**. Se os valores puderem ser negativos, não comece `maior` em 0: inicialize com o valor do primeiro item.

Padrão 3: total e média dos valores

```
precos = {"mouse": 120, "teclado": 200, "monitor": 900, "cabo": 20}

soma = 0

for preco in precos.values():
    soma += preco

media = soma / len(precos)
print(soma, media)

for produto, preco in precos.items():
    if preco > media:
        print(produto, "acima da media")
```

```
1240 310.0
monitor acima da media
```

`len(dicionario)` devolve a quantidade de chaves.

Padrão 4: lista de dicionários (uma "tabela")

Cada dicionário é um registro (um funcionário, um filme, um jogador), e a lista reúne todos.

```
funcionarios = [
    {"nome": "Ana", "salario": 5200},
    {"nome": "Bruno", "salario": 3100},
    {"nome": "Carla", "salario": 6800}
]

melhor = funcionarios[0]
soma = 0

for funcionario in funcionarios:
    soma += funcionario["salario"]

    if funcionario["salario"] > melhor["salario"]:
        melhor = funcionario

print(melhor["nome"], melhor["salario"])
print(soma / len(funcionarios))
```

```
Carla 6800
5033.333333333333
```

Macete

Em uma lista de dicionários, leia os colchetes da esquerda para a direita: `funcionarios[1]["nome"]` é "o funcionário da posição 1" e, dentro dele, "o valor da chave nome". Primeiro a **posição** (lista), depois a **chave** (dicionário). Guardar o **dicionário inteiro** do melhor (e não só o salário) permite exibir qualquer informação dele no final.

Padrão 5: dicionário de listas (agrupar)

Organizar produtos por categoria: a chave é o grupo e o valor é a lista dos integrantes.

```python
produtos = {"mouse": "perifericos", "monitor": "video",
            "teclado": "perifericos"}
grupos = {}

for nome, categoria in produtos.items():
    if categoria not in grupos:
        grupos[categoria] = []

    grupos[categoria].append(nome)

for categoria, nomes in grupos.items():
    print(categoria, nomes, len(nomes))
```

```
perifericos ['mouse', 'teclado'] 2
video ['monitor'] 1
```

Macete

Agrupar é a contagem com outro "valor inicial": na contagem, a chave nova começa com um número; no agrupamento, começa com uma **lista vazia**, e depois vem o `append`. O `if ... not in` só cria a gaveta; o `append` fica **fora** do `if`, porque acontece sempre.

5. Erros mais comuns

Atenção

- **Esquecer as aspas da chave:** `aluno[nome]` procura uma *variável* chamada `nome`; `aluno["nome"]` usa o *texto* nome como chave. Sem aspas só quando a chave está mesmo guardada em uma variável, como em `estoque[produto]`.
- **Acessar sem verificar:** ler ou somar em uma chave que pode não existir. Use `in` antes.
- **Usar in para procurar valor:** ele só olha as chaves.
- **Uma variável só com items():** `for item in d.items()` entrega pares; para separar, use `for chave, valor in d.items()`.
- **Chaves repetidas:** um dicionário não guarda duas chaves iguais. Atribuir de novo **substitui** o valor anterior.
- **Remover durante o percurso:** usar `del` dentro de um `for` sobre o mesmo dicionário causa erro. Guarde primeiro as chaves em uma lista e remova depois.
- **Usar del em chave inexistente:** também causa `KeyError`.

6. Questões de exemplo resolvidas

Questão 1 — Controle de estoque (exercício dos slides)

Crie um dicionário de produtos, consulte a quantidade de um item, altere a quantidade de um produto, adicione um novo produto, remova um produto e percorra tudo exibindo `produto -> quantidade`.

```python
estoque = {"mouse": 10, "teclado": 5, "monitor": 2}

print(estoque["mouse"])

estoque["mouse"] = 8
estoque["cadeira"] = 15
del estoque["monitor"]

for produto, quantidade in estoque.items():
    print(produto, "->", quantidade)
```

```
10
mouse -> 8
teclado -> 5
cadeira -> 15
```

O que observar. A linha `estoque["mouse"] = 8` *altera* (a chave existia) e `estoque["cadeira"] = 15` *adiciona* (não existia). A cadeira aparece por último porque foi a última inserida.

Questão 2 — Letra mais frequente (exercício de fixação 11)

Leia uma palavra e use um dicionário para contar quantas vezes cada letra apareceu. Exiba a letra mais frequente e a quantidade de letras diferentes.

Raciocínio. Padrão 1 (contagem), depois padrão 2 (chave do maior valor). A quantidade de letras diferentes é o tamanho do dicionário.

```python
palavra = "banana"
letras = {}

for letra in palavra:
    if letra in letras:
        letras[letra] += 1
    else:
        letras[letra] = 1

mais_frequente = ""
maior = 0

for letra, total in letras.items():
    if total > maior:
        maior = total
        mais_frequente = letra

print(letras)
print(mais_frequente, maior)
print(len(letras))
```

```
{'b': 1, 'a': 3, 'n': 2}
a 3
3
```

Questão 3 — Filmes em lista de dicionários (exercício de fixação 7)

Armazene filmes em uma lista de dicionários, cada um com título e nota. Exiba o filme com maior nota, o com menor nota e a média das avaliações.

Raciocínio. Padrão 4. O maior e o menor começam apontando para o primeiro filme da lista.

```
filmes = [
    {"titulo": "Horizonte", "nota": 8.5},
    {"titulo": "Orbita", "nota": 6.0},
    {"titulo": "Labirinto", "nota": 9.0}
]

maior = filmes[0]
menor = filmes[0]
soma = 0

for filme in filmes:
    soma += filme["nota"]

    if filme["nota"] > maior["nota"]:
        maior = filme

    if filme["nota"] < menor["nota"]:
        menor = filme

print("Maior:", maior["titulo"])
print("Menor:", menor["titulo"])
print("Media:", soma / len(filmes))
```

```
Maior: Labirinto
Menor: Orbita
Media: 7.833333333333333
```

Questão 4 — Leitura de código

Escreva a saída do programa abaixo.

```
estoque = {"mouse": 10, "teclado": 5}
movimentos = [["teclado", -5], ["cabo", 3], ["teclado", 2]]

for produto, quantidade in movimentos:
    if produto not in estoque:
        estoque[produto] = 0

    estoque[produto] = estoque[produto] + quantidade

    if estoque[produto] == 0:
        del estoque[produto]

    print(len(estoque))

print(estoque)
```

Raciocínio. Acompanhe o dicionário a cada movimento, prestando atenção na **ordem** das chaves.

movimento	o que acontece	dicionário depois
teclado, -5	$5 - 5 = 0$, chave removida	mouse: 10
cabo, 3	chave nova criada com 0, depois $+3$	mouse: 10, cabo: 3
teclado, 2	não existe mais: criada de novo, no **final**	mouse: 10, cabo: 3, teclado: 2

```
1
2
3
{'mouse': 10, 'cabo': 3, 'teclado': 2}
```

A pegadinha: o teclado **voltou**, mas agora depois do cabo, porque foi removido e inserido novamente.

7. Cola rápida

Quero...	**Escrevo...**
criar vazio	`d = {}`
criar com dados	`d = {"nome": "Ana", "nota": 85}`
acessar	`d["nome"]`
alterar ou adicionar	`d["nota"] = 90`
a chave existe?	`"nota" in d`
remover	`del d["nota"]`
quantidade de chaves	`len(d)`
percorrer chaves	`for chave in d:`
percorrer valores	`for valor in d.values():`
percorrer os dois	`for chave, valor in d.items():`
contar ocorrências	`if x in d: d[x] += 1 / else: d[x] = 1`